

Protected Media Containers: Persistent Media Confidentiality under a Compromised Mobile OS

Anonymous Author(s)

Abstract—Mobile applications commonly receive raw camera or microphone data even when their functionality only requires rendering, forwarding, a narrow inference result, or a media transformation. This couples permission to *use* media with permission to *possess its plaintext*, and makes confidentiality depend on the operating system and every application that handles the data. We present *Protected Media Containers* (PMCs), a persistent media abstraction in which plaintext remains confined to protected execution after acquisition, while an untrusted mobile OS may store, route, render, and invoke computation over the object through a restricted interface. We prototype the minimum local PMC lifecycle and define the remote mechanisms as part of the same object model. We evaluate whether useful mobile-media functionality can be preserved while keeping plaintext outside a compromised Android OS.

I. INTRODUCTION

Mobile permission systems commonly equate permission to access a camera or microphone with permission to receive the underlying raw frames or samples. This is stronger than many applications require. A gallery may only need to display an image. A messenger may only need to forward it. A QR reader needs a decoded string. A photo effect may need to transform an image while its output can remain protected. In each case, unrestricted plaintext access is an implementation convenience rather than an application requirement.

The security consequence is significant. Once raw media enters an ordinary application process, a malicious mobile operating system can inspect that process memory, intercept shared buffers, copy files, hook media services, or observe unprotected rendering paths. Application-level permissions therefore provide little confidentiality once the operating system itself is compromised.

We study whether media use can be separated from media possession. Our system treats Android userspace, the Linux kernel, ordinary applications, the filesystem, and the network stack as adversarial. Applications receive handles to persistent *Protected Media Containers* rather than raw image, video, or audio buffers. Android may orchestrate operations over these objects, but cannot directly map the protected plaintext.

The challenge is not simply encryption. A useful mobile abstraction must support several qualitatively different application behaviors. Some applications only need to move or display media. Others require a bounded semantic result. Still others require arbitrary application-specific computation over raw pixels or samples. The same media may then be transferred to another protected client, or deliberately exported to an external service such as a cloud processor or online model. Treating all of these cases as equivalent either exposes too

much information or pushes entire applications and services into the trusted computing base.

These observations motivate our central research question:

Can plaintext possession be separated from media use across local processing, persistent storage, rendering, inter-device transfer, and deliberate release to external services while the mobile operating system itself is malicious?

We answer this question with PMCs, persistent policy-carrying media objects whose plaintext may exist only inside approved protected components unless policy explicitly authorizes release. Locally, the design exposes three operation classes:

- **Content-agnostic operations** such as protected rendering, storage, forwarding, resizing, or rotation produce another PMC or a protected human-visible output, but no application-readable plaintext.
- **Content-inferring operations** invoke fixed trusted primitives such as QR decoding or object recognition and produce a typed result. Such a result is an explicit declassification rather than an implicitly safe derivative.
- **Content-dependent operations** execute arbitrary application-specific logic inside an approved protected module. Their default output is another PMC, not arbitrary normal-world bytes.

Across devices, PMCs support two distinct release modes. In a *protected remote transfer*, the sender releases key material only to an attested protected endpoint, allowing both devices' normal operating systems and the transport application to remain untrusted. In an *authenticated external-service export*, the user intentionally authorizes a trusted service to receive plaintext. The sender authenticates that service, encrypts the PMC for it, and preserves confidentiality only until delivery; what the service does after decryption is outside the PMC guarantee.

The prototype deliberately implements only the minimum local slice required to evaluate the abstraction: PMC creation and persistence, protected rendering, one typed inference primitive, and one protected application-specific PMC-to-PMC transformation. Remote protected transfer and authenticated external-service export remain part of the architecture and threat model, but can be evaluated first through minimal protocol proofs rather than a complete messaging or cloud ecosystem.

This paper makes four contributions:

- [C1] **Persistent protected-media abstraction.** We introduce PMCs, an object model in which encrypted media, sealed metadata, and access policy remain protected across storage, application handoff, rendering, protected computation, and transfer while Android remains untrusted.
- [C2] **Three-way local media interface.** We separate content-agnostic operations, fixed typed declassifications, and arbitrary content-dependent modules whose outputs remain protected by default. This lets applications use media without making raw plaintext access the default capability.
- [C3] **Cross-boundary protected-media-container lifecycle.** We distinguish protection-preserving transfer to an attested protected endpoint from deliberate encrypted export to an authenticated external service. The former extends the PMC plaintext boundary across devices; the latter explicitly terminates it at the authorized service.
- [C4] **Prototype.** We implement the minimum local PMC lifecycle with protected storage, protected rendering, one typed inference workload, and one protected PMC-to-PMC transformation, together with minimal test paths for the two remote release modes.

II. BACKGROUND AND RELATED WORK

A. Opaque Media Handles and Fine-Grained Sensor Access

Darkly is the closest conceptual predecessor to the PMC interface. It replaces OpenCV pointers to raw pixels with opaque references that applications cannot dereference but can pass to trusted image-processing operations [1]. Recognizer similarly argues that applications frequently require semantic features rather than unrestricted raw sensor feeds and exposes higher-level recognizer outputs to augmented-reality applications [2]. Aragorn later sanitizes mobile camera frames so applications receive only intended objects of interest rather than complete images [3].

These systems establish two important principles: applications need not receive raw media by default, and the interface can expose a narrower representation than the source. Their enforcement, however, assumes a trusted platform or camera subsystem. PMCs move the enforcement boundary below Android and make the protected object persist beyond a single sensor-processing call.

B. User-Owned DRM and Protected Media

Personal Mobile DRM demonstrates that users can protect their own media, attach rights, and transfer protected content between phones [4]. Android MediaDrm similarly supports encrypted media whose decryption and playback can be confined to protected components [5]. Prior patent work also applies DRM concepts to user-generated camera and microphone content encrypted close to the imaging source [6].

PMCs differ in the principal trust assumption. The local operating system is not trusted to enforce access policy, and policy governs not only whether content may be consumed but also whether an application receives a protected object, a

typed derivative, access to protected computation, or authority to transfer/declassify the object.

C. Protected Processing and Display

Darkroom processes encrypted images inside Arm TrustZone and returns encrypted results without exposing plaintext to the host operating system [7]. Rushmore securely decrypts and displays static and animated images while leaving the normal world untrusted [8]. Android protected buffers similarly support graphics allocations intended for hardware-protected output paths [9].

Trusted user-interface systems solve a related but distinct problem. TrustUI constructs trusted input and display paths over untrusted mobile drivers [10]. Wink uses TrustZone-controlled I/O and a user-known watermark to distinguish secure from spoofed interaction [11]. PRISM uses Android's Virtualization Framework to protect application widgets and visual interaction despite a compromised kernel [12]. PMCs rely on the existence of such protected output mechanisms, but the contribution is the persistent media object and its operation semantics rather than a new trusted-display primitive.

D. Trusted Sensors and Peripheral Control

Software Abstractions for Trusted Sensors introduced sensor attestation and sensor seal for authentic, policy-bound sensor readings [13]. SGXIO constructs trusted I/O paths despite a malicious operating system [14]. SeCloak enforces TrustZone-backed control over cameras, microphones, radios, and other peripherals despite compromised Android/Linux [15]. TruZ-Droid similarly protects sensitive input and communication through an untrusted mobile stack [16].

Modern Android additionally defines `REQUEST_AVAILABLE_CAPABILITIES_SECURE_IMAGE_DATA`, under which camera images may be written into memory inaccessible to Android userspace and the Android kernel and accessible only to trusted execution environments [17]. Qualcomm documents secure-camera mechanisms, and the MIPI Camera Security Framework specifies camera-link authentication, integrity, and optional encryption [18], [19]. Signing Right Away proposes a TEE-backed camera pipeline for authenticated media provenance [20].

E. TEE Driver Support and Protected Third-Party Execution

A practical trusted peripheral path requires device support below the application abstraction. Driverlets replay a minimal set of device-driver interactions inside TrustZone rather than porting complete commodity drivers [21]. LDR provides a sandboxed runtime for adapted Linux drivers inside embedded TEEs [22]. These systems address the platform-engineering problem beneath PMCs.

Android's Virtualization Framework provides protected virtual machines whose memory can remain confidential from the host Android environment, while protected VM firmware verifies payloads and derives protected identities [23], [24]. Such isolation provides a suitable substrate for application-specific PMC modules without placing every media algorithm directly inside the TEE.

F. Remote Attestation and Authenticated Services

Remote attestation allows a relying party to evaluate signed evidence about a remote protected execution environment before releasing secrets or establishing trust [25]. Android hardware-backed key attestation similarly supports hardware-rooted keys and freshness-bound evidence [26]. PMCs use attestation to distinguish a protected remote recipient from an ordinary remote application.

External services are a different case. If the user intends to submit media to a cloud processor or online model, the service itself is an authorized plaintext endpoint rather than another protected PMC client. In this mode, standard certificate or pinned-key authentication identifies the intended service, while encryption protects the object from the local OS, transport application, and network before delivery.

G. Research Gap

The individual ingredients are established: opaque media references [1], derived sensor interfaces [2], [3], user-owned DRM [4], protected image processing [7], protected display [8], trusted I/O [10], [15], isolated third-party execution [23], and remote attestation [25]. The remaining gap is a persistent mobile media abstraction that composes these ideas while treating Android itself as malicious and that explicitly distinguishes protected use, typed release, arbitrary protected computation, protection-preserving remote transfer, and deliberate external-service declassification.

III. THREAT MODEL AND SCOPE

We consider three protected-media contexts: local use, a remote protected recipient, and an authorized external service.

a) Local adversary.: The adversary controls Android userspace, ordinary applications, system services, the Linux kernel, filesystem, normal-world drivers, shared memory, IPC, and the network stack. It may inspect and modify normal-world memory, forge Android permission state and UIDs, replay requests, modify stored files, substitute endpoints, and invoke PMC interfaces maliciously. Multiple applications may collude.

We trust the hardware root of trust, secure boot chain, TEE or protected hypervisor, protected memory, cryptographic implementation, PMC manager, approved protected modules, and protected rendering/input mechanisms used by the system.

b) Trusted ingress boundary.: Our confidentiality guarantee begins once media has entered protected memory. Secure camera or microphone acquisition is treated as a platform primitive orthogonal to the PMC abstraction; the prototype limitation used to emulate this boundary is described separately in Section V.

c) Policy creation.: Android does not supply an arbitrary PMC policy. At object creation, the PMC manager instantiates an immutable policy from protected configuration based on the object class and authenticated creator or application identity. A normal-world caller may request supported operations but cannot create or widen capabilities, add arbitrary recipients, or authorize arbitrary external services. Destinations not already

present in the protected policy require a trusted policy-update or consent mechanism.

d) Application identity.: Android package identity and UID are not trusted. For the prototype, approved content-dependent modules are provisioned directly into the PMC manager by cryptographic measurement. A production deployment may bind that measurement to a developer signing certificate and a policy describing which PMC interface the module may invoke. Such certification identifies and authorizes code; it does not prove that the algorithm is semantically privacy-preserving.

e) Remote protected recipient.: Another phone or application may receive a PMC while its own Android OS and ordinary application remain malicious. Remote attestation proves that the destination is a genuine approved protected endpoint, but it does not by itself prove that the endpoint is an authorized recipient for this object. The PMC manager therefore requires both (i) an accepted attestation bound to the destination session key and (ii) a recipient identity already authorized by the PMC's immutable policy. The transport application and both devices' normal-world network stacks may observe and relay ciphertext but cannot recover plaintext.

If the recipient TEE, protected hypervisor, secure boot root, attestation key, or accepted protected module is itself compromised while continuing to produce evidence accepted by the verifier, the guarantee cannot be maintained. Such compromise is outside our threat model.

f) Authorized external service.: A user may deliberately export media to an external service such as an online vision model, language model, or cloud processor. This is an explicit declassification boundary rather than a protection-preserving PMC transfer. Certificate or pinned-key authentication proves the identity of the service, but not that the user intended to release this PMC to it. The PMC manager therefore exports only to service identities already authorized by the object's immutable policy. It authenticates the service, encrypts the media for that service, and exposes only ciphertext to the local OS, requesting application, transport stack, and network. We do not require attestation of the service's internal software stack. Once the authenticated service decrypts the media, its storage, processing, administrators, downstream components, and subsequent use of plaintext are outside our scope.

g) Out of scope.: Denial of service, invasive physical attacks, microarchitectural and power side channels, analog re-recording of rendered media, compromise of the local or remote TEE/protected hypervisor itself, complete traffic-analysis resistance, and covert channels from protected modules through timing, output size, resource consumption, or other externally observable behavior are outside scope. Android may observe object existence, coarse ciphertext size, access timing, and network endpoints.

TABLE I

POSITIONING AGAINST REPRESENTATIVE RELATED SYSTEMS. CHECKS INDICATE CAPABILITIES DIRECTLY PROVIDED BY THE CITED SYSTEM UNDER ITS STATED THREAT MODEL.

System	Compromised OS tolerated	Trusted sensor ingress	Raw media hidden from normal app	Typed / filtered release	Persistent protected object	Protected custom computation	Protected rendering	PMC-to-PMC default output	Protected remote transfer	External-service export	Application-facing lifecycle abstraction
Personal DRM [4]	×	×	✓	×	✓	×	✓	×	✓	×	×
Darkly [1]	×	×	✓	×	×	✓	×	×	×	×	×
Recognizer [2]	×	×	✓	✓	×	×	×	×	×	×	×
Darkroom [7]	✓	×	✓	×	✓	✓	×	✓	×	×	×
SeCloak [15]	✓	×	✓	×	×	×	×	×	×	×	×
Rushmore [8]	✓	×	✓	×	✓	×	×	×	×	×	×
Aragorn [3]	×	✓	✓	×	×	×	×	×	×	×	×
PRISM [12]	✓	×	—	×	×	✓	✓	×	×	×	×
PMC (ours)	✓	A	✓	✓	✓	✓	✓	✓	✓	✓	✓

A denotes trusted sensor ingress assumed as a platform primitive rather than implemented by PMC. The table compares capabilities relevant to the PMC problem rather than implying identical goals across systems.

IV. DESIGN

V. EVALUATION

VI. DISCUSSION AND LIMITATIONS

VII. CONCLUSION

REFERENCES

- [1] S. Jana, A. Narayanan, and V. Shmatikov, “A scanner darkly: Protecting user privacy from perceptual applications,” in *2013 IEEE Symposium on Security and Privacy*, 2013, pp. 349–363.
- [2] S. Jana, D. Molnar, A. Moshchuk, A. Dunn, B. Livshits, H. J. Wang, and E. Ofek, “Enabling fine-grained permissions for augmented reality applications with recognizers,” in *22nd USENIX Security Symposium*. USENIX Association, 2013, pp. 415–430. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity13/technical-sessions/presentation/jana>
- [3] H. Venugopalan, Z. A. Din, T. Carpenter, J. Lowe-Power, S. T. King, and Z. Shafiq, “Aragorn: A privacy-enhancing system for mobile cameras,” *Proceedings of the ACM on Interactive, Mobile, Wearable and Ubiquitous Technologies*, vol. 7, no. 4, 2024.
- [4] S. Bhatt, R. Sion, and B. Carburnar, “A personal mobile drm manager for smartphones,” *Computers & Security*, vol. 28, no. 6, pp. 327–340, 2009.
- [5] Android Developers, “MediaDrm api reference,” <https://developer.android.com/reference/android/media/MediaDrm>, accessed 2026.
- [6] G. Clift, “Computer ecosystem providing privacy and tracking in sharing user-generated content by encrypting the ugc at the imaging source,” U.S. Patent 9,152,806, 2015. [Online]. Available: <https://patents.google.com/patent/US9152806B2/en>
- [7] T. Brito, N. O. Duarte, and N. Santos, “ARM TrustZone for secure image processing on the cloud,” in *2016 IEEE 35th Symposium on Reliable Distributed Systems Workshops*, 2016, pp. 37–42.
- [8] C. M. Park, D. Kim, D. V. Sidhwani, A. Fuchs, A. Paul, S. J. Lee, K. Dantu, and S. Y. Ko, “Rushmore: Securely displaying static and animated images using TrustZone,” in *Proceedings of the 19th Annual International Conference on Mobile Systems, Applications, and Services*, ser. MobiSys ’21, 2021, pp. 122–135.
- [9] Android Open Source Project, “Bufferqueue and gralloc: Protected buffers,” <https://source.android.com/docs/core/graphics/arch-bq-gralloc>, accessed 2026.
- [10] W. Li, M. Ma, J. Han, Y. Xia, B. Zang, C.-K. Chu, and T. Li, “Building trusted path on untrusted device drivers for mobile devices,” in *Proceedings of the 5th Asia-Pacific Workshop on Systems*, ser. APSys ’14, 2014.
- [11] A. Chakraborti, D. Suci, and R. Sion, “Wink: Deniable secure messaging,” in *32nd USENIX Security Symposium*. USENIX Association, 2023, pp. 1271–1288. [Online]. Available: <https://www.usenix.org/conference/usenixsecurity23/presentation/chakraborti-wink>
- [12] Y. Ng, Z. Chen, H. Qiu, and X. Ding, “PRISM: To fortify widget based user-app data exchanges using android virtualization framework,” in *Proceedings of the 20th ACM Asia Conference on Computer and Communications Security*, 2025, pp. 1567–1581.
- [13] H. Liu, S. Saroiu, A. Wolman, and H. Raj, “Software abstractions for trusted sensors,” in *Proceedings of the 10th Annual International Conference on Mobile Systems, Applications, and Services*, ser. MobiSys ’12, 2012, pp. 365–378.
- [14] S. Weiser and M. Werner, “SGXIO: Generic trusted i/o path for intel SGX,” in *Proceedings of the Seventh ACM Conference on Data and Application Security and Privacy*, ser. CODASPY ’17, 2017, pp. 261–268.
- [15] M. Lentz, R. Sen, P. Druschel, and B. Bhattacharjee, “SeCloak: ARM TrustZone-based mobile peripheral control,” in *Proceedings of the 16th Annual International Conference on Mobile Systems, Applications, and Services*, ser. MobiSys ’18, 2018.
- [16] K. Ying, A. Ahlawat, B. Alsharifi, Y. Jiang, P. Thavai, and W. Du, “TruZ-Droid: Integrating TrustZone with mobile operating system,” in *Proceedings of the 16th Annual International Conference on Mobile Systems, Applications, and Services*, ser. MobiSys ’18, 2018.
- [17] Android Developers, “CameraMetadata.REQUEST_AVAILABLE_CAPABILITIES_SECURE_IMAGE_DATA,” <https://developer.android.com/reference/android/hardware/camera2/CameraMetadata>, accessed 2026.
- [18] Qualcomm Technologies, Inc., “Camera security,” <https://www.qualcomm.com/products/features/mobile-security/camera-security>, accessed 2026.
- [19] MIPI Alliance, “Mipi camera security framework,” <https://www.mipi.org/specifications/mipi-camera-security>, accessed 2026.
- [20] Y. Jang, “Signing right away,” 2025, whitepaper. [Online]. Available: <https://arxiv.org/abs/2510.09656>
- [21] L. Guo and F. X. Lin, “Minimum viable device drivers for ARM TrustZone,” in *Proceedings of the Seventeenth European Conference on Computer Systems*, ser. EuroSys ’22, 2022, pp. 300–316.
- [22] H. Yan, Z. Ling, H. Li, L. Luo, X. Shao, K. Dong, P. Jiang, M. Yang, J. Luo, and X. Fu, “LDR: Secure and efficient linux driver runtime for embedded TEE systems,” in *Network and Distributed System Security Symposium*, 2024.
- [23] Android Open Source Project, “Android virtualization framework (AVF) overview,” <https://source.android.com/docs/core/virtualization>, accessed 2026.
- [24] —, “Android virtualization framework: Security,” <https://source.android.com/docs/core/virtualization/security>, accessed 2026.
- [25] H. Birkholz, D. Thaler, M. Richardson, N. Smith, and W. Pan, “Remote attestation procedures (RATS) architecture,” IETF, Tech. Rep. RFC 9334, 2023. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc9334.html>
- [26] Android Developers, “Verify hardware-backed key pairs with key attestation,” <https://developer.android.com/privacy-and-security/security-key-attestation>, accessed 2026.